

Plano de ação — oportunidades de produto vs. RedTrack.io

Plano de produto derivado da pesquisa de mercado da **RedTrack.io** (jun/2026). Traduz as recomendações P0/P1 em **fases de implementação ancoradas no código atual** do Orion Agency. Contexto e fontes: ver a pesquisa de mercado (resumo no histórico desta pasta) e o [README do Cérebro](#) + [pesquisa-e-concorrentes](#).

TL;DR

- A RedTrack **não é concorrente direto** — é produto vizinho (tracking/atribuição multicanal server-side + anti-fraude). Nosso fosso continua sendo **Meta-first + PT-BR + agência + Agency Brain** (memória IA cross-cliente/cross-nicho), que ela **não tem**.
- Não vamos** virar plataforma de tracking multicanal nem anti-fraude standalone (longe do nicho, alto custo). Isso fica **fora de escopo** por ora.
- 3 frentes priorizadas**, todas alinhadas ao que já temos:
 - P0 — Meta CAPI (Conversions API)**: envio server-side de conversões para a Meta. Maior impacto/esforço; melhora direta de performance pra quem roda Meta Ads. Aproveita `meta-graph.ts`, tokens em `meta-auth-store.ts` e o pixel que já manipulamos em `meta-promoted-object.ts`.
 - P1 — Agente de IA / MCP sobre o Agency Brain**: transformar a memória/insights num agente acionável (chat → ações) e expor via **MCP server**. É a tendência dominante do mercado e já temos o contexto estruturado pronto em `get-client-brain-context.ts` + `/api/agency-brain/chat`.
 - P2 — Modelos/janelas de atribuição dentro do Meta**: valor de "atribuição" sem construir tracking genérico.

Por que estas três (e não as outras)

Capacidade RedTrack	Vale pra nós?	Decisão
Meta CAPI / conversions API	✅ Direto no nosso core (Meta Ads)	P0 — fazer
AI agent + MCP sobre os dados	✅ Temos o ativo único (Agency Brain)	P1 — fazer
Modelos/janelas de atribuição	🟡 Parcial, só dentro do Meta	P2 — fazer enxuto
Tracking multicanal server-side	❌ Fora do nicho, alto custo	Não fazer agora
Anti-fraude standalone	❌ Fora do nicho	Não fazer agora

Sinal de mercado que valida P1: a RedTrack **monetiza IA como tier pago** — AI Team **\$499/mo** (MCP + 5 AI dashboards), overage **\$29/dashboard/mo**, add-on **Premium MCP \$199/mo**. Triple Whale lançou "Moby Agents" (abr/2025). Mas media buyers ainda veem agentic AI como "inevitável, porém incremental", travando em **acurácia e transparência** — então nosso diferencial deve ser **explicabilidade** (citar o aprendizado/nicho que embasou cada sugestão).

P0 — Meta CAPI (Conversions API)

Objetivo: enviar eventos de conversão server-to-server para a Meta (`POST /{pixel-id}/events`), recuperando sinal perdido por iOS/ad blockers e melhorando a otimização das campanhas dos clientes.

O que já temos (reaproveitar):

- Cliente Graph v20.0 em `meta-graph.ts` (com `metaFetchWithRateLimit`).
- Tipo `MetaPixel` e descoberta de pixel já existentes no fluxo Meta.
- Tokens por tenant em `meta-auth-store.ts` (`getAllTenantMetaTokens`).
- Mapeamento de evento de conversão/pixel no lado da campanha em `meta-promoted-object.ts` (`custom_event_type` / `custom_conversion_id`).
- Rate-limit/erros: `meta-rate-limit.ts` , `meta-graph-errors.ts` .

Fases:

- **P0.1 — Cliente CAPI + envio manual/teste**
 - Novo `src/lib/meta-capi.ts` : `sendConversionEvent({ pixelId, token, event })` → `POST {GRAPH_BASE}/{pixelId}/events` com `data: [{ event_name, event_time, action_source, user_data (hasheado), custom_data }]` + suporte a `test_event_code` .
 - **Hashing de PII obrigatório** (SHA-256 de email/telefone normalizados) antes do envio — a Meta exige; reusar util de hash ou criar `src/lib/meta-capi-hash.ts` . (A própria RedTrack destaca "hashing de PII on-the-fly" — é requisito, não diferencial.)
 - Persistir config por cliente: `pixelId` + `capiToken` (ou reusar token do sistema) — provável nova coluna/JSON em `Client` ou em `client-meta-settings` .
- **P0.2 — Deduplicação com o Pixel do browser**
 - Gerar/propagar `event_id` para deduplicar com o pixel client-side (evita conversão dupla). Documentar que o cliente precisa enviar o mesmo `event_id` no pixel do navegador.
- **P0.3 — Eventos a partir dos nossos dados**
 - Mapear conversões que já lemos das insights (`actions` / `results` em `MetaInsightRow`) e/ou webhooks do cliente para eventos CAPI. Decidir a fonte do evento (form/checkout do cliente vs. reenvio de offline conversions).
- **P0.4 — UI + entitlement + observabilidade**
 - Toggle por cliente em Configurações/Integrações; status "CAPI ativo / eventos enviados (24h) / último erro".
 - Gate por plano (seguir o padrão `assertLimit` de `billing/entitlements`).

- Log de eventos enviados/recusados (a Meta retorna `events_received` + `messages`).

Riscos/decisões:

- **De onde vêm os eventos de conversão?** Sem um pixel/endpoint do cliente, CAPI server-side puro fica limitado a *offline/reenvio*. Definir isto **antes** de P0.3 (é o que determina o valor real).
- Token: usar token do usuário (OAuth) vs. system user token — impacta permissão/escala.
- Janela de match real é ~10–45% do sinal perdido (não "100%"); comunicar expectativa honesta.

Critério de pronto: um cliente com pixel configurado dispara um evento de teste (`test_event_code`) visível no Events Manager da Meta, com dedupe por `event_id` e PII hashado.

P1 — Agente de IA / MCP sobre o Agency Brain

Objetivo: evoluir do chat read-only atual para um **agente** que lê o Agency Brain e **propõe/ executa ações** (pausar perdedor, escalar vencedor, criar público), e **expor os dados via MCP** para ferramentas de IA externas — com **explicabilidade** como diferencial.

O que já temos (reaproveitar):

- Contexto estruturado pronto "para prompts de IA e **agentes futuros**": `get-client-brain-context.ts` (learnings aprovados, DNA, top criativos, tags, summaryText).
- Chat já funcional (read-only) em `/api/agency-brain/chat` com gate de billing (`allowAgencyBrainChat`) e uso de IA (`geminiGenerateJson`).
- Motor de automação condição→ação já existe: `automation-engine.ts` e `/automations` — **as "ações" do agente devem reusar este motor**, não reinventar.
- Recomendações: `/api/ai/recommendations`.

Fases:

- **P1.1 — Tools/ações nomeadas (function calling)**
 - Definir um catálogo de *tools* que o agente pode chamar (ex.: `getClientBrain`, `listCampaigns`, `pauseCampaign`, `scaleBudget`, `createLookalike`), cada uma mapeada para serviços que **já existem** (insights, `automation-engine`, `meta-audience-create`).
 - Toda ação **mutável** passa por **confirmação humana** (proposta → aprovar) — alinhado ao fluxo de curadoria do Cérebro e à barreira de confiança do mercado.
- **P1.2 — Explicabilidade**
 - Cada sugestão cita **qual learning/sinal/nicho** a embasou (já temos `evidence` nos learnings e `comparedTo` de mercado). Esse é o nosso diferencial vs. "caixa-preta".
- **P1.3 — MCP server (read-only primeiro)**
 - Servidor MCP expondo recursos do Brain (learnings, DNA, métricas, criativos) como *resources/ tools* read-only, autenticado por tenant. Espelha o `getClientBrainContext`.
 - **Monetizar como tier** (espelhando RedTrack): MCP/agente atrás de um add-on ou plano superior, com limite de requisições — encaixar no billing por planos/add-ons que já temos.
- **P1.4 — MCP read/write + cadência**

- Habilitar ações de escrita via MCP (com a mesma confirmação humana) e definir cadência/limites.

Riscos/decisões:

- **Confiança:** ação automática sem revisão é o maior risco reputacional. Default = **propor, não executar** até o cliente optar por automação.
- **Stack de IA:** hoje usamos **Gemini** (`geminiGenerateJson`). Function calling/agente pode pedir reavaliação do provedor/modelo — decisão a registrar.
- Escopo do MCP: começar **read-only** reduz risco e já entrega o valor de "conectar minha IA aos meus dados".

Critério de pronto (P1.1): no chat, pedir "o que pausar essa semana?" retorna **propostas acionáveis com botão de aprovar** que, ao confirmar, executam via `automation-engine` e citam o learning que as embasou.

P2 — Modelos/janelas de atribuição dentro do Meta

Objetivo: deixar o usuário escolher **janela** (7d/30d/...) e **modelo** de atribuição sobre os dados que já lemos da Meta, sem construir tracking server-side genérico.

- Aproveitar `MetaInsightRow.actions` / `purchase_roas` e os parâmetros de `action_attribution_windows` da Graph API nas chamadas de insights (`meta-graph.ts` / `meta-insights-cache.ts`).
- UI: seletor de janela nos relatórios/dashboard; persistir preferência por tenant/cliente.
- **Escopo enxuto:** é uma melhoria de leitura/relatório, não um novo subsistema. Priorizar só depois de P0 e do P1.1.

Fora de escopo (decisão explícita)

- **Tracking multicanal server-side** (Google/TikTok/etc. estilo RedTrack/Voluum): alto custo, fora do nicho Meta-first. Reavaliar só se entrarmos forte em multicanal.
- **Anti-fraude standalone:** idem. (O hashing de PII do P0.1 é requisito da CAPI, não anti-fraude.)

Sequência sugerida

1. **P0.1–P0.2** (CAPI + dedupe) — base de valor imediato e vendável.
2. **P1.1–P1.2** (agente com ações + explicabilidade) — diferencial de produto.
3. **P0.3–P0.4** (eventos reais + UI/billing da CAPI).
4. **P1.3** (MCP read-only, monetizado).
5. **P2** (atribuição) e **P1.4** (MCP read/write) conforme tração.

Perguntas em aberto (a resolver antes de codar)

- **CAPI:** qual a fonte real dos eventos de conversão do cliente (pixel/endpoint próprio, checkout, offline)? Sem isso, o valor da CAPI fica limitado a reenvio offline.
- **Brasil:** a RedTrack tem tração real no mercado BR? Se não, reforça PT-BR como fosso e o posicionamento "RedTrack para agências BR de Meta".
- **Monetização de IA:** agências BR pagariam um tier "AI Team" (estilo \$499/mo + \$199 MCP) ou o agente deve ser feature inclusa nos planos atuais?
- **Provedor de IA:** manter Gemini para function calling/agente ou avaliar alternativa?

Premissas / limites desta análise

- Fatos da RedTrack vêm majoritariamente das **páginas oficiais deles** (marketing) — bons para "o que oferecem", não para eficácia comprovada. Preços capturados em **25/jun/2026** e **voláteis** (houve renomeação recente de planos) — reconferir antes de usar em material.
- Entre os concorrentes citados, **só a Triple Whale** foi verificada a fundo na pesquisa; Voluum, Hyros, Northbeam, Cometly, ClickMagick **não** foram aprofundados.
- O comparativo de features do Orion baseia-se no estado do código em jun/2026 (arquivos referenciados acima); reconfirmar in-repo antes de decisões de roadmap.

Histórico

- 2026-06-25: Documento criado a partir da pesquisa de mercado da RedTrack.io — recomendações P0 (Meta CAPI), P1 (agente/MCP sobre o Agency Brain) e P2 (atribuição no Meta), com fases ancoradas no código atual. Tracking multicanal e anti-fraude marcados como fora de escopo.